هياكل البيانات

المدة
٢٠ دقيقة

المواد
هاتف وعليه هذا الدرس بعد تنزيله، كومة كتب أو أطباق لتمثيل المكدّس (اختياري)

أهداف التعلّم

- تختار قائمة أو مكدّسًا أو طابورًا أو مجموعة أو قاموسًا حسب العمليات التي تحتاجها المهمة.
- تستعمل مكدّسًا لفحص الأقواس.
- تعدّ بقاموس.

النشاط: مكدّس الأقواس

١. التمهيد (٣ دقائق): ضع ثلاثة كتب فوق بعضها ثم خذها: آخر ما وُضع أول ما يُؤخذ. وسمِّ ميزة في الهاتف تعمل هكذا.
٢. الفحص (٧ دقائق): اقرأ «([{}])» حرفًا بعد حرف. وضع كل قوس فاتح على كومة من قصاصات الورق، وعند كل قوس مغلق خذ القصاصة العليا وتحقّق من أنها تطابقه.
٣. الكسر (٧ دقائق): الفاحص الساذج يعدّ فقط أن عدد (يساوي عدد). فاكتب سلاسل تخدعه، مثل «)(»، وافحصها بمكدّسك.
٤. الكود (٣ دقائق): شغّل سلاسلك على balanced() في الهاتف. هل التقطها المكدّس كلها؟

قصاصات الورق مكدّس مثالي: لا يمكن أخذ إلّا العليا.

للنقاش

- لماذا يكون فحص المجموعة أسرع بكثير من فحص القائمة؟
- متى يكون الطابور اختيارًا خاطئًا؟
- ماذا تتخلّى عنه المجموعة مقابل سرعتها؟

تحقّق من نفسك

- سمِّ الهيكل المناسب لكلٍّ من: زر الرجوع في المتصفّح، ومهامّ الطابعة، وعدّ الكلمات.

جرّبها هذا الأسبوع

عُدّ حروف اسمك الكامل بقاموس: لكل حرف count[c] = count.get(c, 0) + 1. أيّ حرف يظهر أكثر؟

ورقة عمل

هياكل البيانات

البرمجة، المستوى ٤، الدرس ٣

الاسم _____ الفصل _____ التاريخ _____

١ **يجب أن يطبع برنامج الطلاب بترتيب تسجيلهم، وأن يفحص أيضًا آلاف المرات إن كان رقم ما مسجّلًا. ماذا يحفظ؟**

◯ قائمة للترتيب ومجموعة للفحص ◯ مجموعة فقط ◯ طابور فقط

٢ **اكتب balanced(text) بمكدّس: تعيد True إن أُغلق كل قوس من ( ) [ ] { } بشريكه بالترتيب الصحيح، وFalse في غير ذلك. وتُتجاهل الرموز الأخرى.**

```
def balanced(text):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    # push each opening bracket;
    # at each closing one, pop and check its partner
    return True
```

اكتب برنامجك:

٣ **املأ الفراغات: يعدّ البرنامج الأصوات بقاموس ويطبع عدد أصوات Ali، أي 2.**

```
votes = ["Ali", "Bo", "Ali"]
count = {}
for v in votes:
    count[v] = count.__________(v, __________) + 1
print(count["Ali"])
```

بنك الكلمات: `add` `get` `1` `0`

٤ **ماذا يطبع هذا البرنامج؟**

```
q = ["Amina", "Kofi", "Layla"]
q.append("Omar")
first = q.pop(0)
print(first)
```

◯ Omar ◯ Layla ◯ Amina

٥ **صِل كل مهمة بالهيكل الأنسب.**

تاريخ التراجع	• • • •	طابور
زبائن يُخدَمون بترتيب وصولهم	• • • •	قاموس
هل هذا الرقم مسجّل؟	• • • •	مكدّس
رقم هاتف طالب بحسب اسمه	• • • •	مجموعة

فكّر في نظام في مدرستك أو جامعتك (المكتبة، أو المقصف، أو الجدول). ما العمليات التي يؤدّيها أكثر، وأيّ الهياكل ستختار؟

تحقّق من إجاباتك

هياكل البيانات

١ قائمة للترتيب ومجموعة للفحص

تجيب المجموعة عن «هل هو موجود؟» في خطوة تقريبًا، لكنها لا تحفظ ترتيبًا يمكن الاعتماد عليه، والقائمة أو الطابور يحفظان الترتيب، لكنهما يفحصان واحدًا واحدًا.

٢

```
def balanced(text):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for c in text:
        if c in "([{":
            stack.append(c)
        elif c in pairs:
            if not stack or stack.pop() != pairs[c]:
                return False
    return len(stack) == 0
```

يجب أن يُغلق أحدث قوس فاتح أولًا: آخر داخل أول خارج، أي مكدّس.

٣

```
votes = ["Ali", "Bo", "Ali"]
count = {}
for v in votes:
    count[v] = count.get(v, 0) + 1
print(count["Ali"])
```

تعطي count.get(v, 0) العدد حتى الآن، أو 0 لاسم يظهر أول مرة، وإضافة 1 تسجّل هذا الصوت.

٤ Amina

تنضمّ append من الخلف وتخرج pop(0) من المقدّمة: طابور، أول داخل أول خارج.

٥ تاريخ التراجع — مكدّس

زبائن يُخدَمون بترتيب وصولهم — طابور

هل هذا الرقم مسجّل؟ — مجموعة

رقم هاتف طالب بحسب اسمه — قاموس

الأحدث أولًا، والأقدم أولًا، و«هل هو موجود؟»، والبحث بالمفتاح: أربع عمليات وأربعة هياكل.